Business 41204

Machine Learning

More Supervised Learning

Outline:

1. Classification
2. In the Woods
3. Stacking and AutoML
4. Interpreting “Black Box” Learners

1. Classification

Classification

Classification = jargon for predicting a categorical outcome

- Predict default (yes/no)
- Predict churn (yes/no)
- Predict animal type in image (cat/dog/…/other)

Same basic concepts apply to classification and regression

- validation exercises to choose good performing models
- standard performance metrics for validation differ from standard performance metrics for regression
  - Regression metrics still make sense though
- many classification algorithms return predicted class probabilities
  - Some just return predicted class though

Classification Evaluation

Typically MSE, MAE not used for evaluating classification performance

- Can be used though and do make sense

Baseline Evaluation Measures:

- Accuracy – fraction of time prediction is correct
  - Extremely simple and intuitive
  - Directly targets object that is often of interest
  - Coarse – not uncommon that many prediction rules give same accuracy

- Cross-entropy (or log) loss

$$L\big(Y, f(X)\big) = -\sum_{c} 1(Y = c)\log\big(f_c(X)\big)$$

  - Y has $c = 1, \dots, K$ classes, prediction $f(X) = \big(f_1(X), \dots, f_K(X)\big)$ returns predicted probability for each class (denoted $f_c(X)$)
  - I.e. looking at how well model/predicted probabilities line up with data
  - Numerically - not terribly meaningful to most people (hard to interpret)
  - Less "coarse" than accuracy as it focuses on probabilities rather than the discrete predictions

Classification Evaluation

There are other common measures that look more specifically at true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN)

Further Evaluation Measures:

- Precision = TP/(TP + FP) = TP/(total positive predictions)
  - high when there are few FP
  - care about this when FP are costly (e.g. spam detection, fraud detection)
- Specificity = TN/(TN + FP) = TN/(total negatives in data)
  - High when there are few FP
  - 1 – Specificity = false positive rate
- Recall (aka sensitivity, true positive rate) = TP/(TP + FN) = TP/(total positives in data)
  - high when there are few FN
  - care about this when FN are costly (e.g. medical screening)
- F1 = 2TP/(2TP + FP + FN)
  - aims to “balance” FP and FN

Code for Bank Deposit Example

Bank Example

Goal: Predict whether a person subscribes to a bank term deposit in response to a telemarketing campaign

Data:

- n = 41188 customers
- p = 19 predictors
  - include demographic and macroeconomic variables
- S. Moro, P. Cortez and P. Rita (2014) “A Data-Driven Approach to Predict the Success of Bank Telemarketing. Decision Support Systems,” *Decision Support Systems* 62, 22-31.

- Use 80% for training, 20% validation

Baseline Models

Two baseline models:

1. Logistic regression with raw predictors
   - Accuracy = 0.900
   - Cross-entropy = .2803
2. Constant (i.e. ignore predictors)
   - Accuracy = 0.885
   - Cross-entropy = .3565

Ignoring all predictors is almost as accurate as logistic regression. What do we care about?

Why is ignoring the predictors so accurate?

Confusion Matrix

The confusion matrix shows prediction results by displaying TP, FP, TN, FN.

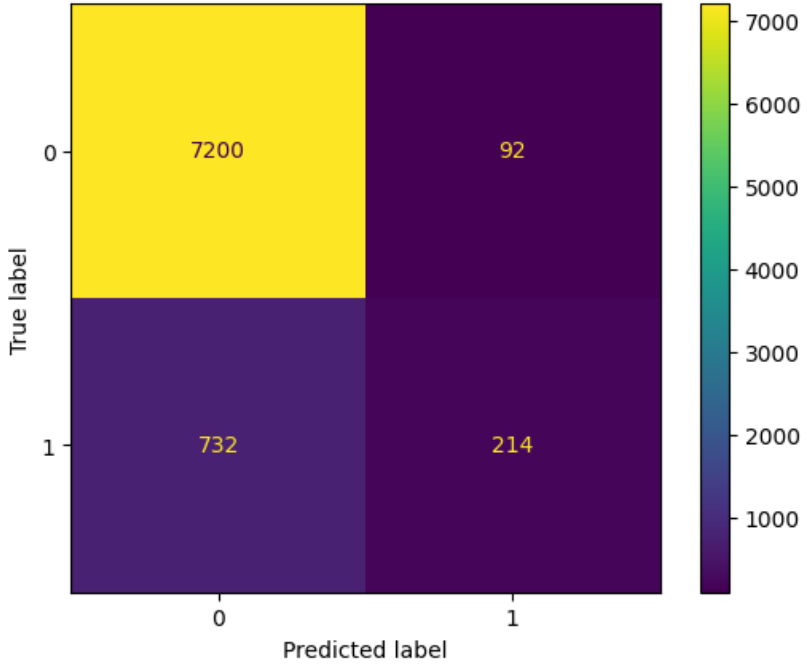
For logistic regression:
TP (214), FP (92),
TN (7200), FN (732).

Accuracy:
0.900 = (7200+214)/8238

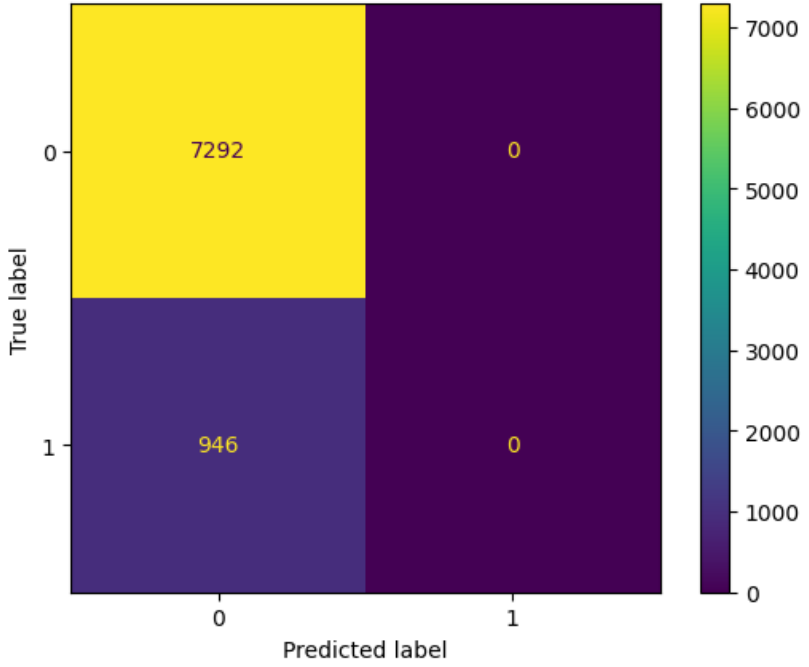
correct predictions

total predictions

Logistic regression confusion matrix



Constant prediction confusion matrix



Even though we get similar accuracy, the prediction rules are interestingly different.

Other Common Accuracy Statistics

Given the TP, FP, TN, FN from the confusion matrix, can compute precision, specificity, recall, and f1.

	Y=1	
	Constant	**Logistic**
precision	0	.699
recall	0	.226
specificity	1	.987
f1-score	0	.342

Not well-defined for constant model: 0/0

Precision = TP/(TP + FP) = 214/(214+92) = .699

Recall = TP/(TP + FN) = 214/(214+732) = .226

Specificity = TN/(TN + FP) = 7200/(7200+92) = .987

f1 = 2TP/(2TP + FP + FN) = 2*214/(2*214+92+732) = .342

ROC and AUC

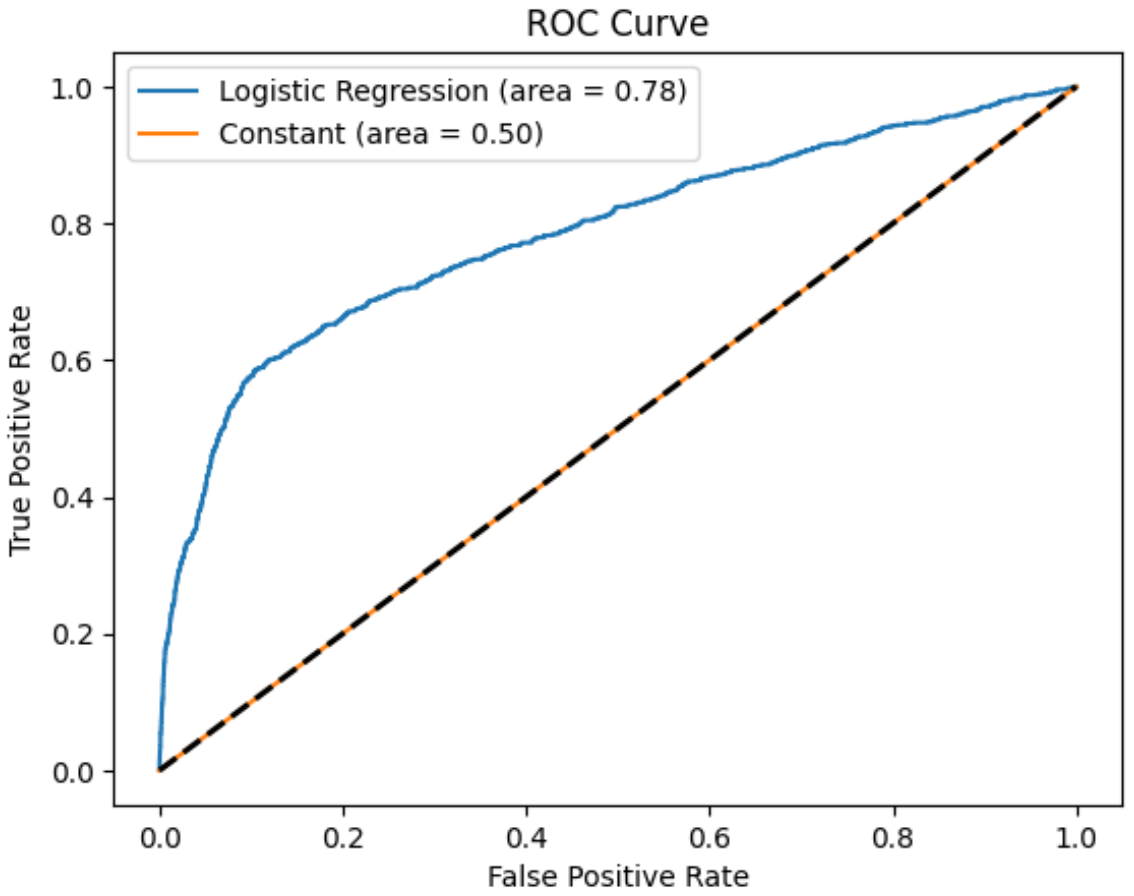
ROC is another popular summary of classification quality

ROC plots

$$TPR = TP/(TP + FN) \text{ vs } FPR = FP/(FP + TN)$$

by varying decision threshold between 1 and 0

- when decision threshold is 1, all classified as 0 so no FP or TP
- as threshold decreases, start getting FP and TP
- random guessing gives equal FP and TP (on average)
- perfect classification would be all TP and no FP



AUC – area under the ROC curve

- best possible is 1
- random guessing is .5

Cumulative Gain

Thought experiment:

- Contact X% of customers/population
- See how many positives you get

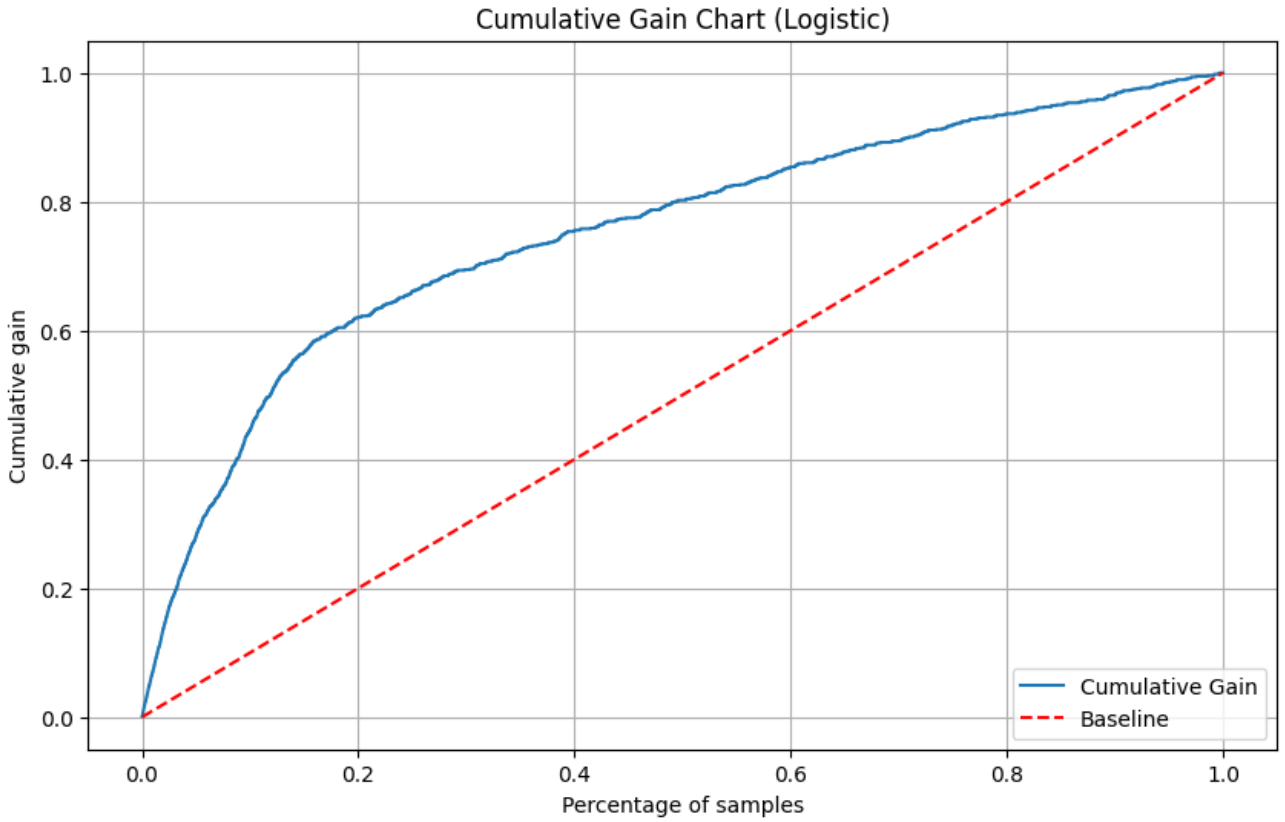
X-axis – fraction of sample “contacted”
Y-axis – fraction of true positives “reached”

Baseline – contact at random

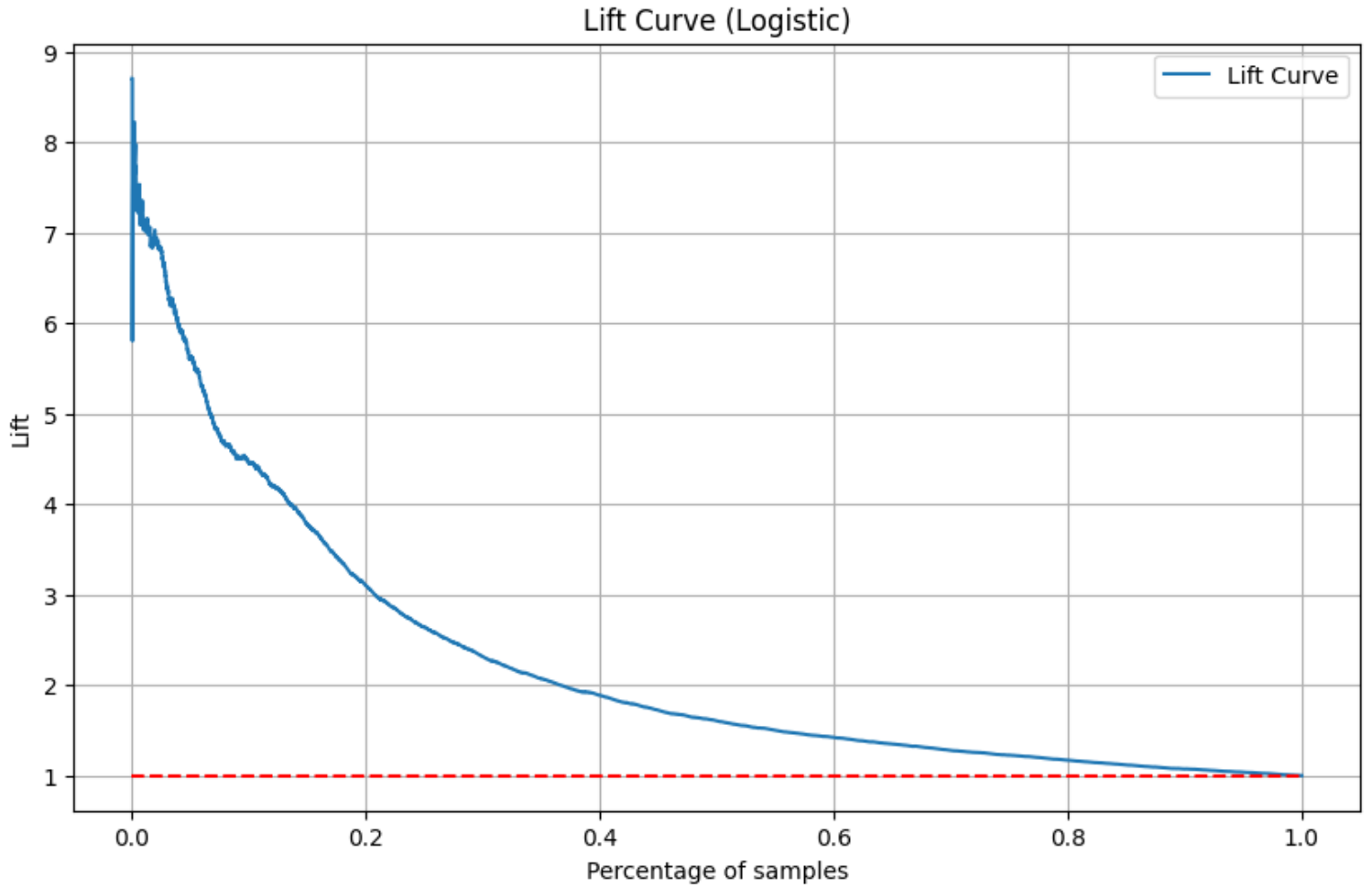
- e.g. would expect to get 10% of total positives if contact 10% at random

Gain – rank observations according to predicted probability, “contact” in order of ranking

- shows gain of using predictive model



Lift



Lift: Simply

$$\frac{\text{Gain}}{\text{Baseline}}$$

Shows how much more likely to reach positive class by using model than by “contacting” at random

2. In the Woods

“Modern” Learners

“Modern” learners aim to automate feature construction while providing high quality approximations

- Available for regression and classification

Tree-based methods are a popular class of modern learners

- CART (classification and regression trees)
- random forests
- boosted trees

The other popular class is (deep) neural networks (LATER)

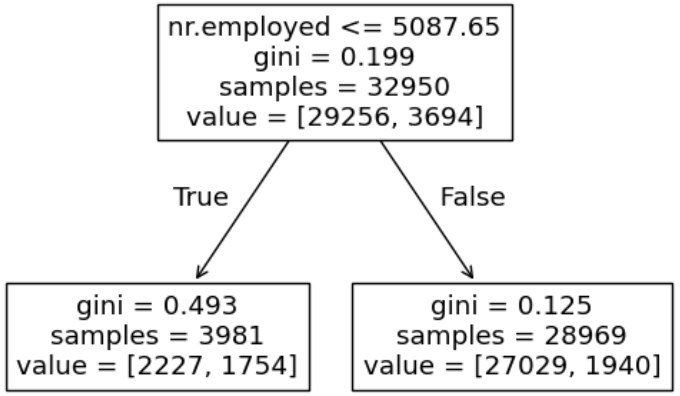
Trees

Basic idea of tree-based methods:

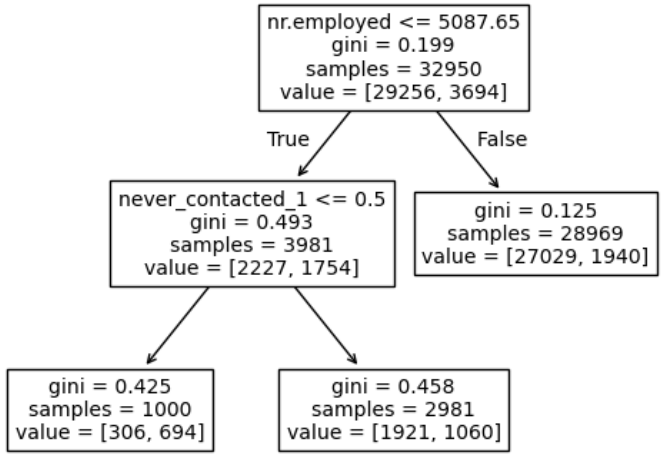
- Adaptively partition the feature space into regions
  - Usually rectangles
- Fit a simple prediction rule within each region
  - Usually a constant (e.g. the sample mean within the region)
- Partition and prediction rule jointly chosen to get good in-sample fit
  - Combination of choosing partition and prediction rule means trees are doing representation learning
  - Can choose to never partition on a feature – means this feature is "dropped"
  - In practice, just need to specify the (raw) features and target

Let's just see what trees look like in the bank example.

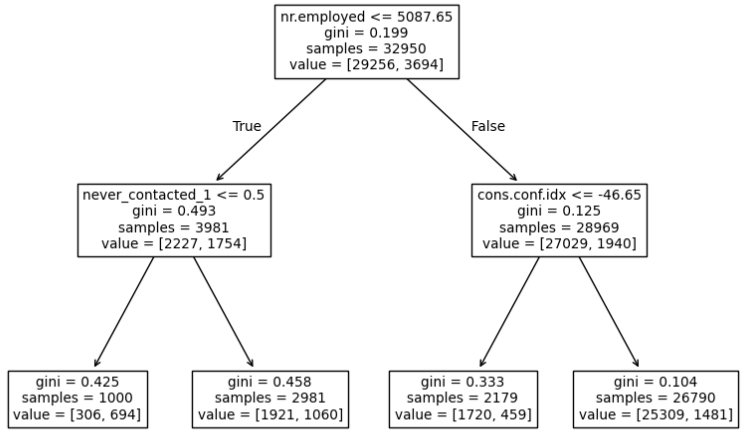
Some Tree Models



2 leaves;
depth 1



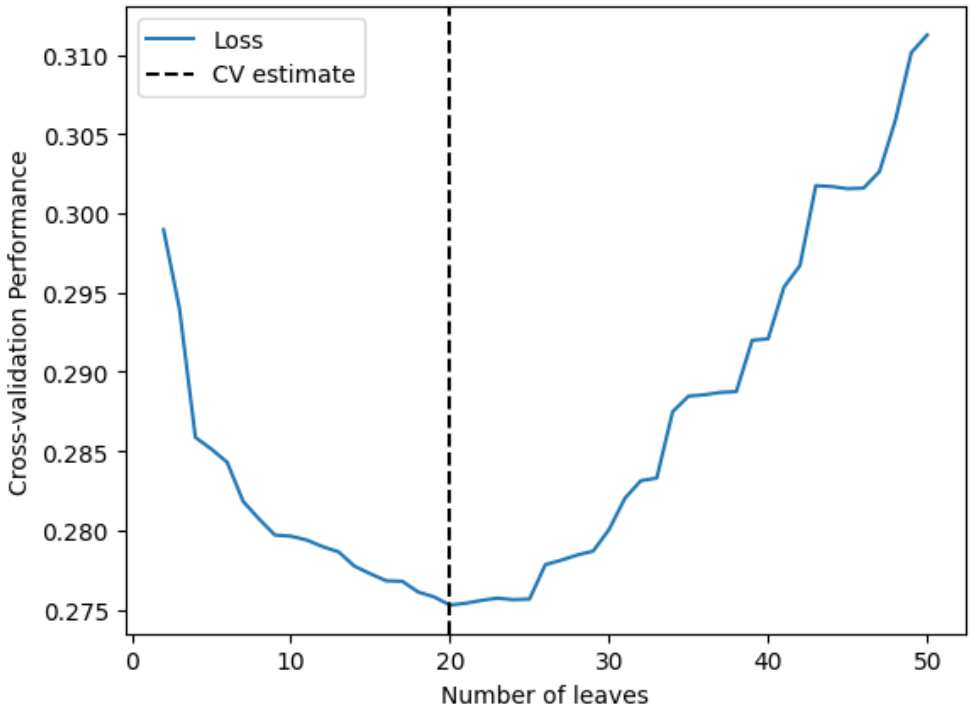
3 leaves;
depth 2



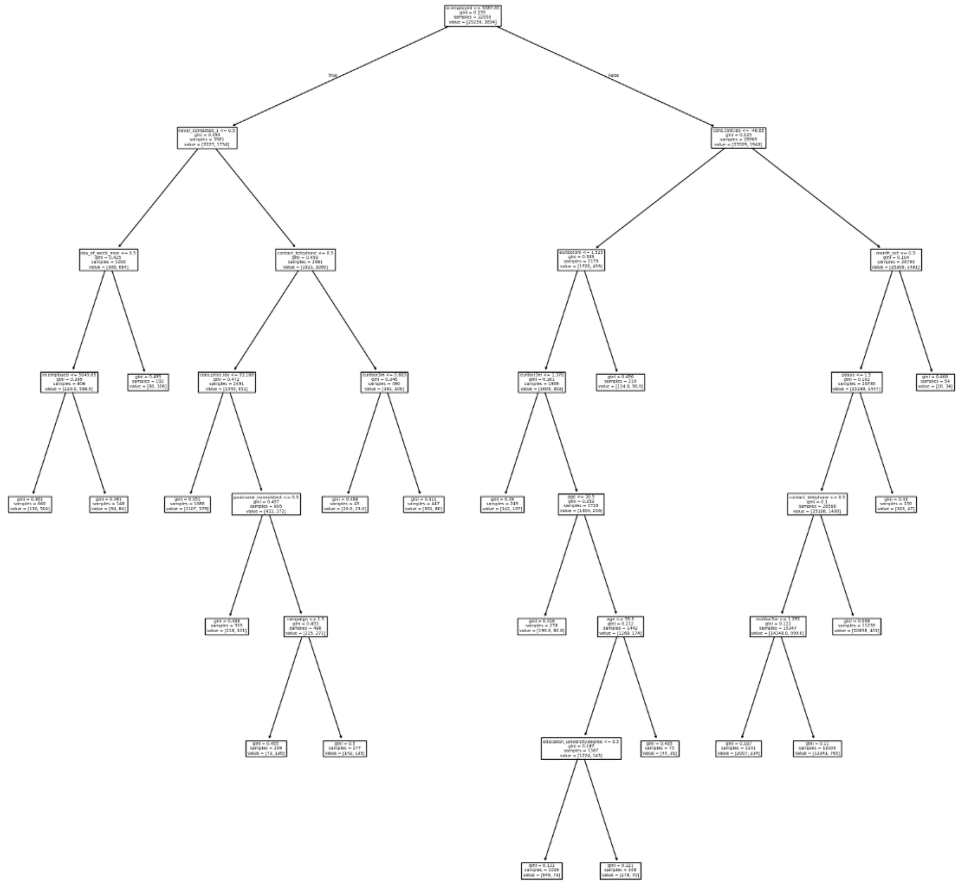
4 leaves;
depth 2

How do we choose?

Cross-validation over leaves



Using *cross-entropy* loss



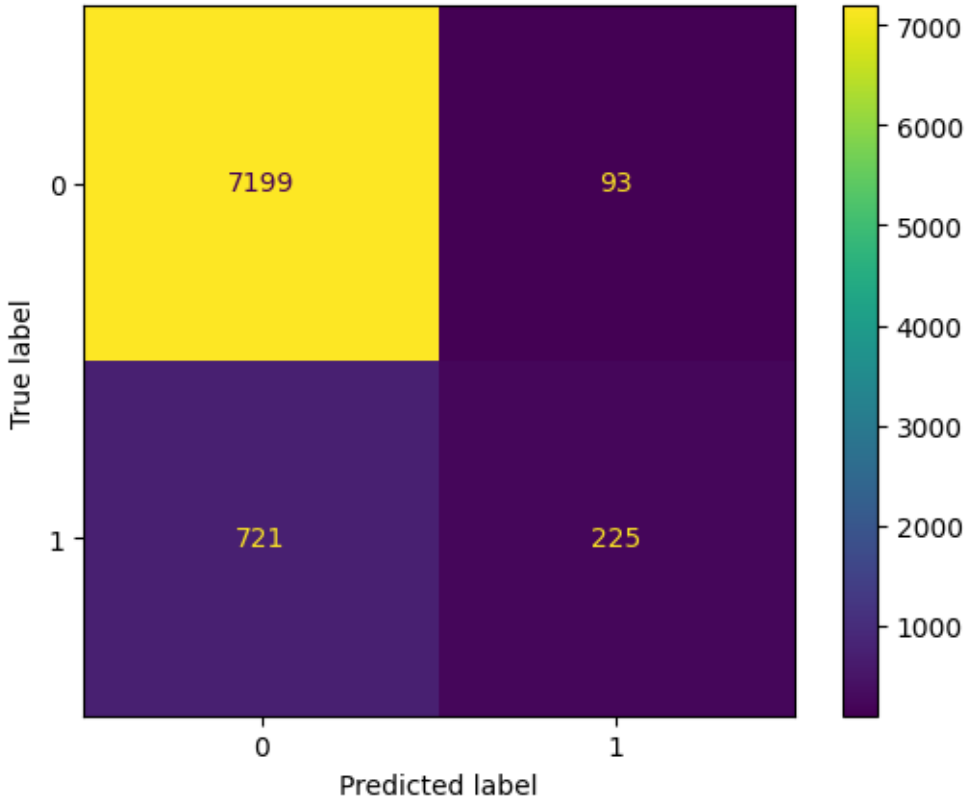
Tree selected using cross-validation

(We'd need a pretty big screen to make sense of it)

Validation Performance

For fun, let's try a tree as well.

	Y=1		
	Constant	**Logistic**	**Tree**
precision	0	.699	.708
recall	0	.226	.238
specificity	1	.987	.987
f1-score	0	.342	.356



Accuracy = .901
Cross-entropy = .276

Random Forest

Random forest:

- based on a large number of trees (a forest)
- each tree is trained using random data (random)
  - Two main approaches:
    - Take a random subsample of the available training data
    - Take a random sample ***with replacement*** from the training data
  - Typically also take a random subset of the available predictors
- Special case of a powerful idea called "bagging" due to L. Breiman (1996) "Bagging predictors" *Machine Learning* 26, 123-140.

In math:

- Let b denote the b^{th} randomly generated set of data
- Let $\widehat{tree}_b(x)$ be the tree prediction rule learned using data set b
- The random forest prediction rule is $\widehat{rf}(x) = \frac{1}{B}\sum_{b=1}^{B} \widehat{tree}_b(x)$ where B is the total number of estimated trees

Some “Intuition” for Random Forests

Why should random forests work?

- Remember, trying to make “bias/variance” tradeoff
  - What would happen if all the trees were unbiased and independent?
  - Easy to fit “low-bias” trees – Use lots of leaves
- Will trees fit on different data produce the same splits?
  - What happens if you average step functions with steps at different points?

In principle, lots of tuning choices for random forests

- tend to work well “out-of-the-box” though
- part of the sales pitch is relative insensitivity to tree specific choices
  - use a large B
  - use “big” trees

Still judging quality based on out-of-sample prediction performance

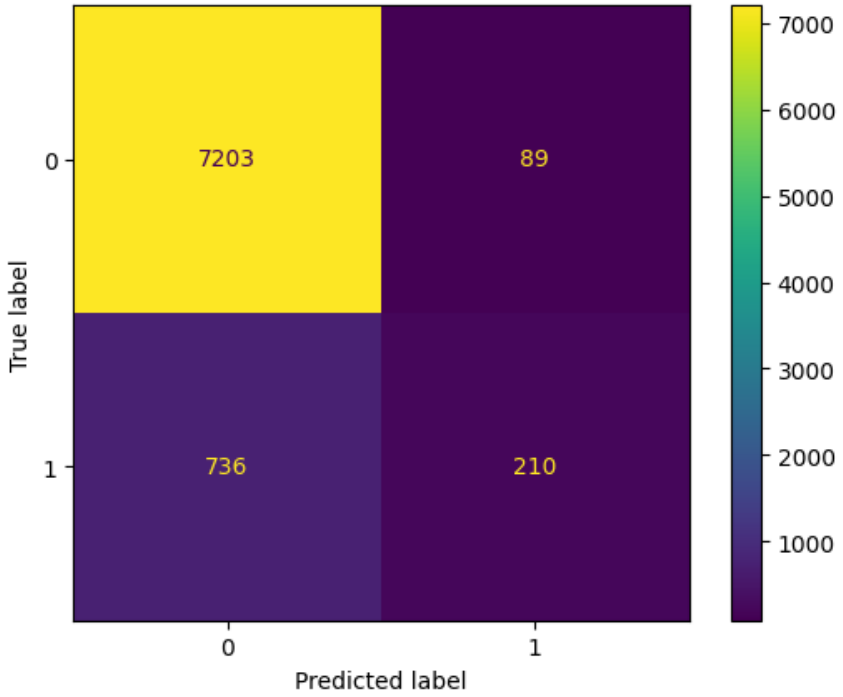
Bank: Random Forest

CV:

Min Leaf Size	Number of Trees	CV Loss
1	100	0.496343
1	250	0.385055
1	500	0.331215
1	1000	0.314575
15	100	0.27215
15	250	0.271963
15	500	0.271842
15	**1000**	**0.271781**
30	100	0.272634
30	250	0.272589
30	500	0.272623
30	1000	0.272596
60	100	0.274422
60	250	0.274324
60	500	0.27434
60	1000	0.274243
120	100	0.276618
120	250	0.276515
120	500	0.27642
120	1000	0.276354

Best performance
(but most are pretty close)

Validation Performance



Accuracy = .900
Cross-entropy = .274

	Ran. Forest	Logistic	Tree
precision	.702	.699	.708
recall	.222	.226	.238
specificity	.988	.987	.987
f1-score	.337	.342	.356

Boosting

Boosting:

- basic idea is to improve model fit in small increments by *recursive fitting*
  - Fit a simple model and obtain prediction errors
  - Fit a new simple model to the prediction errors and construct new prediction errors
  - Repeat …
  - Final prediction rule is sum of all prediction rules
- each step must improve fit (within training data) relative to previous step
  - overfit if too many steps taken – typically do (cross-)validation to choose number of steps
- popular with tree methods
  - R. E. Schapire. The strength of weak learnability. Machine Learning, 5:197–227, 1990.

Boosted Trees

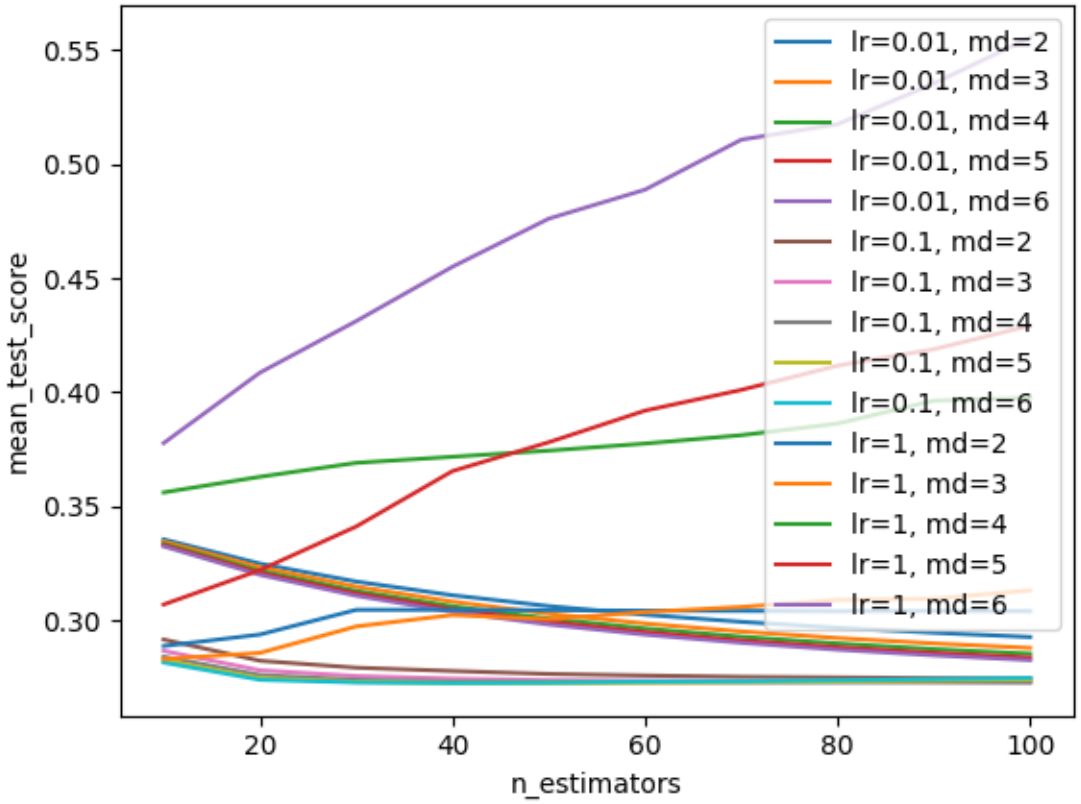
Basic boosting algorithm for trees with data $\{y_i, x_i\}_{i=1}^n$:

1. Initialize *residuals* (prediction errors) as $r_{i,1} = y_i$
2. For b = 1,…,B
   1. Fit tree based prediction rule $\widehat{tree}_b(x)$ using data $\{r_{i,b}, x_i\}_{i=1}^n$
      - Should be a simple tree. E.g. a tree of depth d for d small
   2. Update the residuals to $r_{i,b+1} = r_{i,b} - \lambda \widehat{tree}_b(x_i)$
      - λ is called the learning rate and plays an important role. Usually want $\lambda \ll 1$. (We'll see learning rates again with deep neural networks)
3. Construct the final prediction rule as $\widehat{boost}(x) = \sum_{b=1}^B \lambda \widehat{tree}_b(x)$

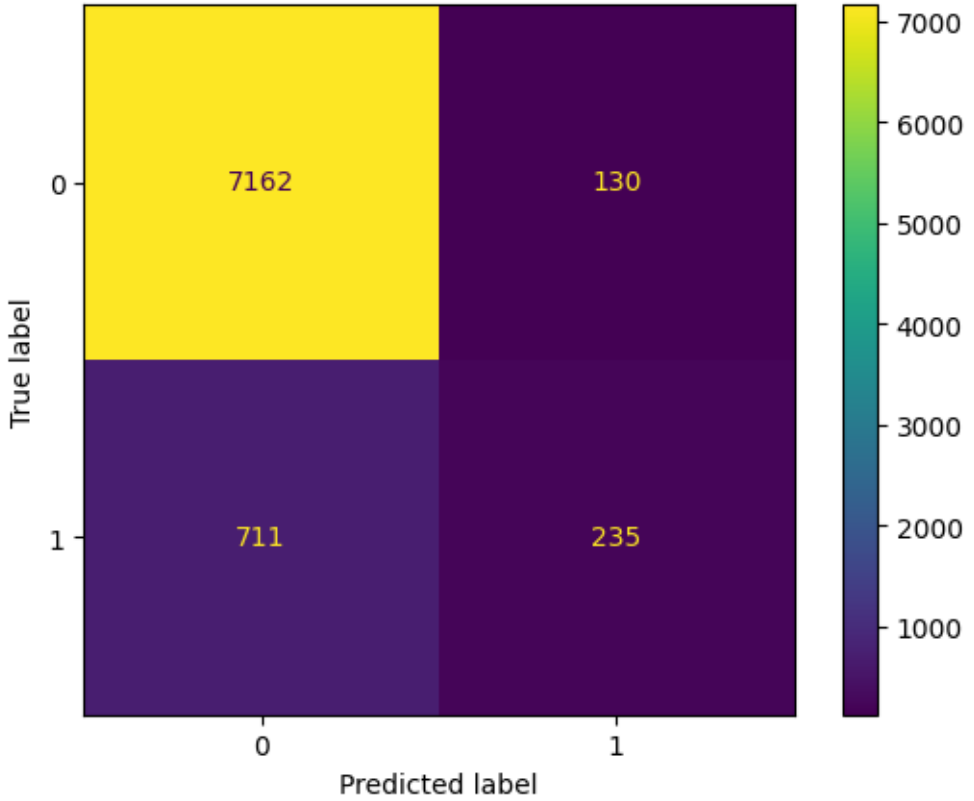
Tuning parameters - λ, B, d (or other tree parameters) chosen by (cross-)validation

Bank: Boosted Trees

CV:



Validation Performance



Accuracy = .898
Cross-entropy = .275

	GBC	Ran. Forest	Tree
precision	.644	.702	.708
recall	.248	.222	.238
specificity	.988	.988	.987
f1-score	.359	.337	.356

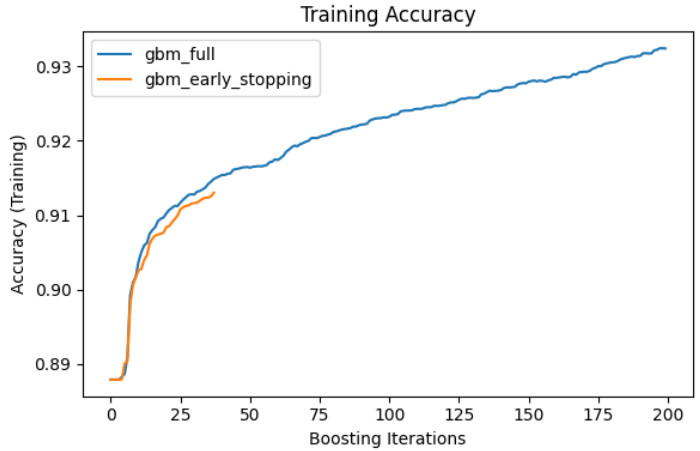
Early Stopping

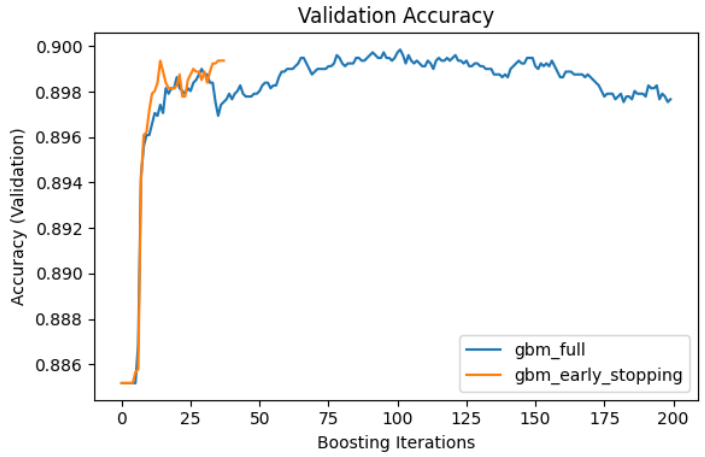
Many learners (including boosting)

- are updated in a step by step fashion
- are such that beyond a certain number of steps, training fit will continue improving but out-of-sample performance is likely to degrade
- instead of training a fixed number of steps, can monitor validation performance and stop when improvement levels off/worsens

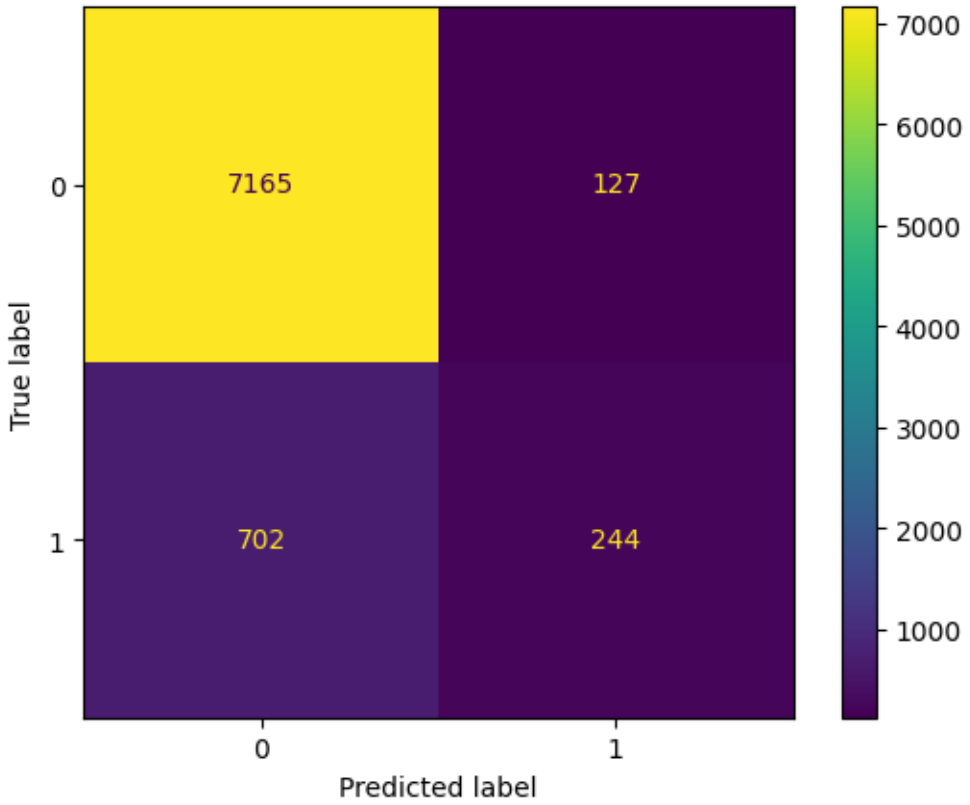
= ***Early Stopping***

Can save computation time AND improve prediction performance!





Bank: Boosted Trees with Early Stopping



Accuracy = .899
Cross-entropy = .275

	GBC-ES	GBC	Ran. Forest	Tree
precision	.658	.644	.702	.708
recall	.258	.248	.222	.238
specificity	.983	.988	.988	.987
f1-score	.371	.359	.337	.356

“Expected Profit”

Might want to use “economic” criterion to choose model

Things we might think about in stylized setting for determining the lifetime value of a contact resulting in a new account:

1. Initial Deposit and Balance Maintenance over time
   - Capital the bank can use for loans and investments
2. Interest and Fee Income
   - Interest Revenue generated from the customer's account balance
   - Fee Income from the customer such as account maintenance fees, overdraft fees, and transaction fees
3. Cross-Selling Opportunities
4. Customer Retention and Longevity
   - Average Customer Lifespan/Churn Rate
5. Cost of Acquisition and Servicing
   - Costs associated with acquiring a new customer, including marketing and sales expenses.
   - Cost of servicing the account, including customer service and account management.
6. Discount Rate

We’re not going to consider most of these in our little example.

Stylized Bank Deposit Revenue

Let's make up some numbers for use in our example.

- Suppose cost C = 100 of contacting an individual
- Suppose customers who open accounts are all the same
  - Initial deposit of 1000, 2000 average maintained balance
  - Assessed fees 50/year
  - Take out credit card that generates 100/year in fees and interest
  - Stay with bank for 5 years
- Suppose the bank
  - Earns 5%/year on loans made from deposits
  - Incurs 100 in initial costs and 20/year for account maintenance, setup, …
  - Has a discount rate of 5%

Net customer revenue from account opener is then 130 in year 1 and 230 in years 2-5.

- Gives LTV of customer of ≈ 900

Bank Deposit Benefit

We can then calculate "expected" benefit of contacting someone we predict will open account:

$$PR(TP)(900 - 100) + PR(FP)(-100)$$

where we use our prediction models to fill in estimates for the true positive and false positive rates.

Model	Estimated "Expected" Profit
Constant	0.0
Logistic	19.66
Tree	20.72
Random Forest	19.31
Boosted Tree	21.24
Boosted Tree (Early Stopping)	22.15

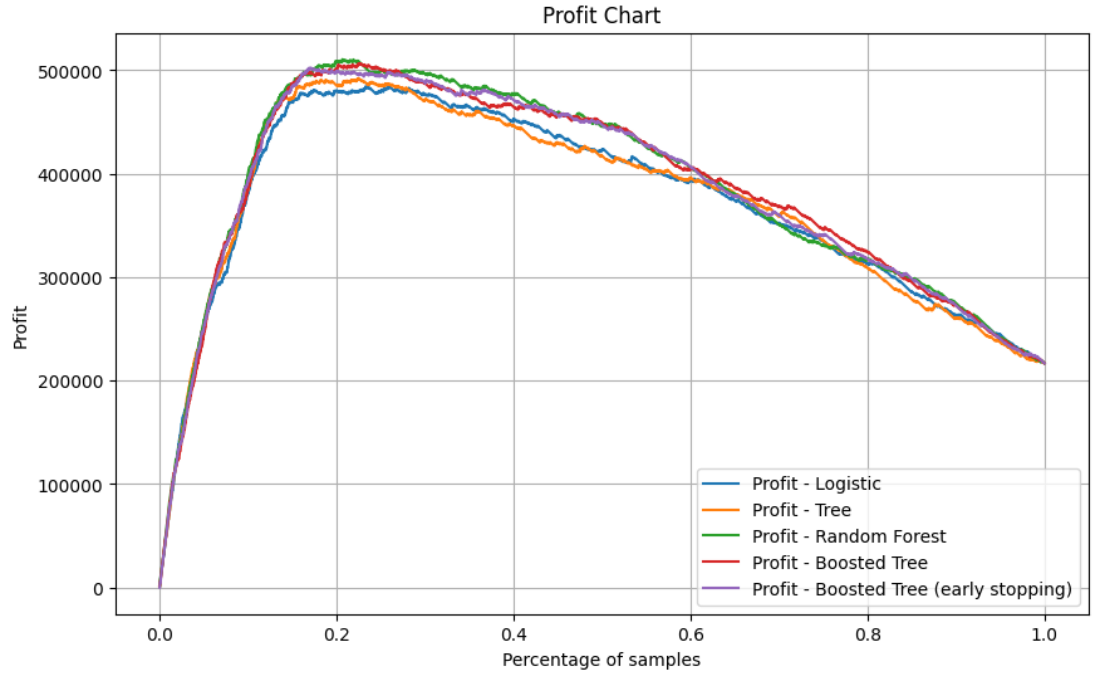
Using a rule that contacts if predicted take-up probability $\geq 50\%$

Why is this 0?

Bank Deposit “Profit” Chart

A different exercise, akin to ROC, is to construct a “profit” chart:

1. First, order all individuals in the sample according to predicted probability of take-up.
2. Consider varying the threshold for contact (or alternatively the fraction of the sample to contact) between 1 (contact no one) and 0 (contact everyone)
3. Choose the model and threshold that gives the highest "profit." (All the true ones give a benefit of R-C, and all the true zeros give a benefit of -C.)



Model	Threshold	“Profit”	Accuracy
Logistic	0.103	484500	0.801
Tree	0.104	490300	0.808
Random Forest	0.109	510600	0.829
Boosted Tree	0.105	508300	0.831
Boosted Tree (Early Stopping)	0.174	494100	0.874

Accuracy is lower than baselines (≈ .9)! Do we care?

3. Stacking and AutoML

Model Choice

Hard (impossible?) to know the right model to try in any given setting

- Already seen this in choosing tuning parameters via (cross-)validation

Should try several models and see which does best

- BUT, different models may perform better/worse for different instances

Stacking = Rather than choose one model, *combine models* to leverage strengths of each!

AutoML = Use machines to automate the whole model building process

- Try different learning approaches (e.g. random forests, boosting, neural networks …)
- Try different configurations of tuning parameters
- Combine models

Stacking

Stacking is a simple model combination idea.

Suppose you have $\mathrm{m} = 1, \ldots, M$ "base" models for predicting outcome Y

- Models trained using cross-validation
  - Let $\hat{Y}_i^m$ be the (out-of-sample/cross-validation) prediction for observation i based on model m
- Stacking takes $(\hat{Y}_i^1, \ldots, \hat{Y}_i^m)$ to use as predictors for building a *meta*-model for predicting Y
  - For regression tasks, common implementation is to use $(\hat{Y}_i^1, \ldots, \hat{Y}_i^m)$ to predict Y using linear regression
    - Can also used penalized linear regression – especially useful if m is large
  - For classification tasks, common implementation is to use $(\hat{Y}_i^1, \ldots, \hat{Y}_i^m)$ to predict Y using (multinomial) logistic regression
    - Can also used penalized (multinomial) logistic regression – especially useful if m is large
  - I.e. we just treat the (out-of-sample) predictions produced by our different candidates as data that can be used to predict the outcome

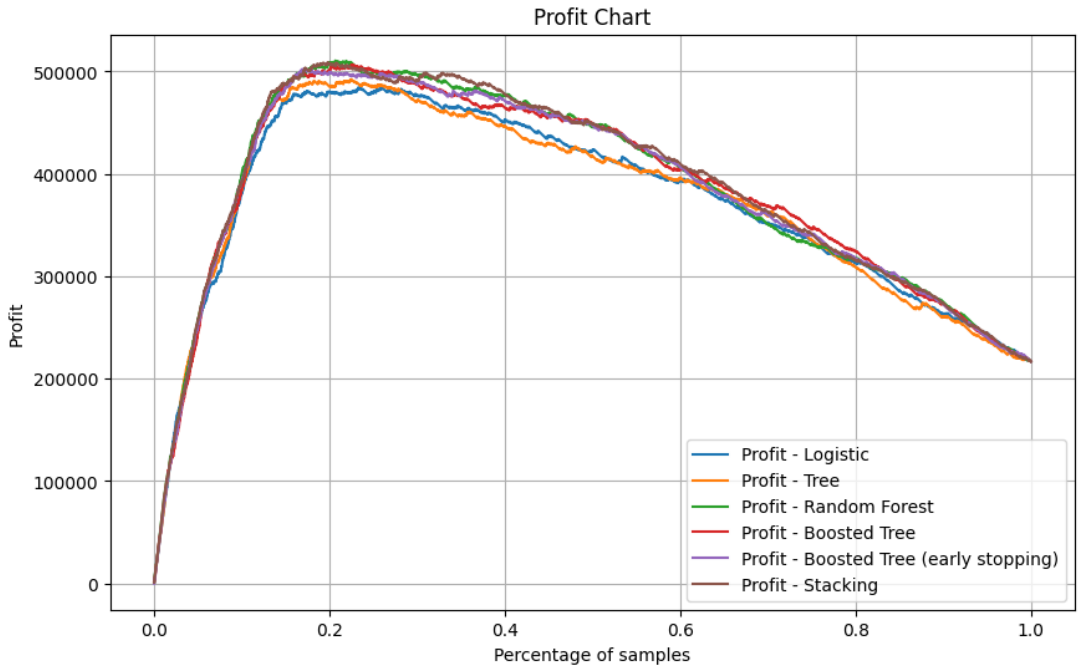
Stacking in Bank Example

Let's look at stacking in the bank deposit example using the learners from the previous sections.

- Combine the learners using linear regression

Model	Logistic Regression Coefficient
Logistic	0.041
Tree	0.094
Random Forest	0.557
Boosting	0.342
Constant	-0.004

Stacking accuracy: 0.899
Cross-entropy: 0.273



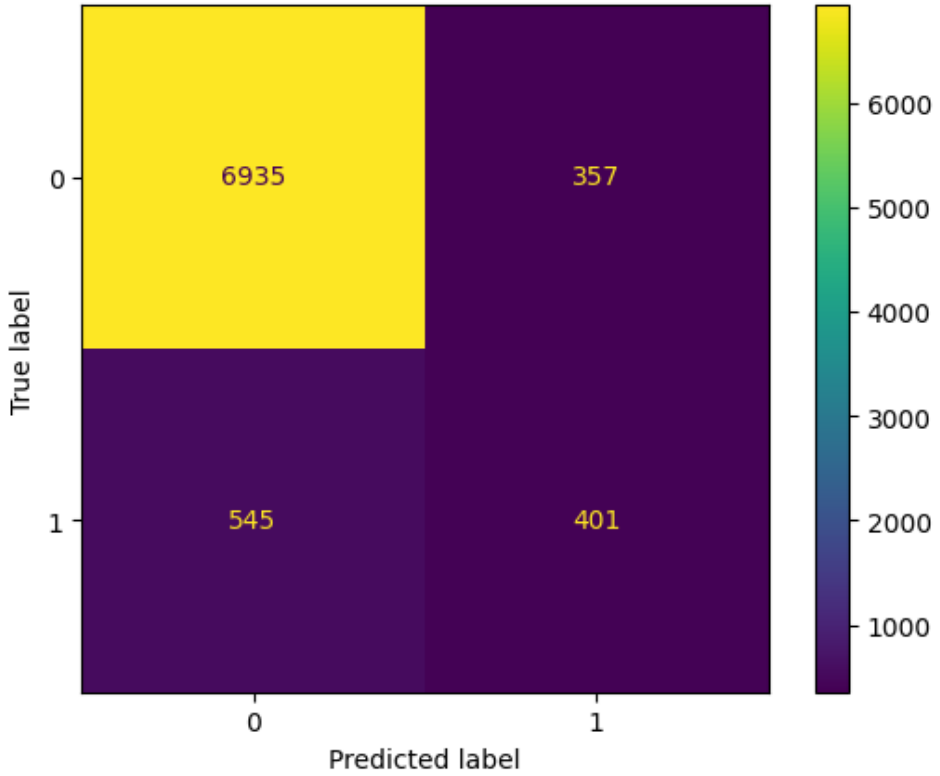
AutoML

The basic idea of automatic machine learning (AutoML) is straightforward:

- We're already using a computer to try lots of models
- Wrap the whole thing in a package that just automates going through models
- Ideally, a user would just indicate the target variable and predictor variables and the machine would do the rest
  - Not quite there in my experience, but getting closer all the time

AutoML: H2o makes use of a set number of models (set here to 20).

- Accuracy = 0.891
- Cross-entropy = 0.276
- Predicts many more "1"s than other approaches



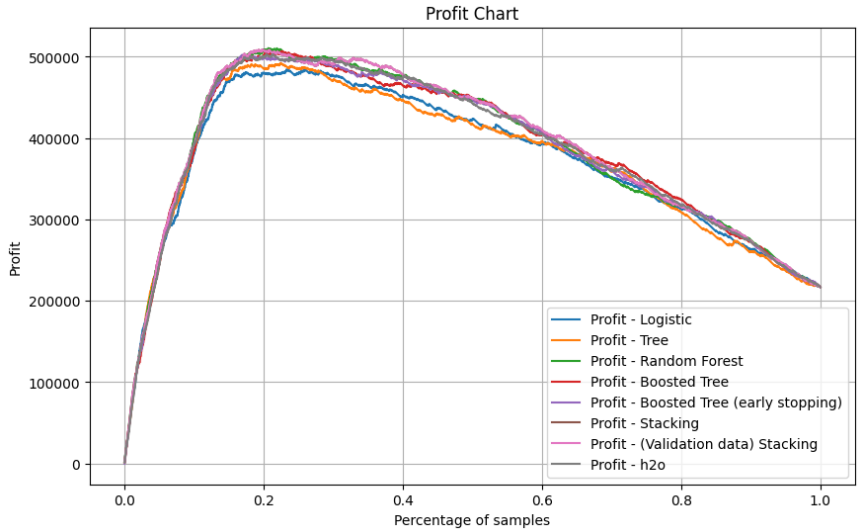
AutoML

Classification Metrics

	h2o	GBC-ES	GBC	Ran. Forest	Tree	Logistic
precision	.529	.658	.644	.702	.708	.699
recall	.424	.258	.248	.222	.238	.226
specificity	.951	.983	.988	.988	.987	.987
f1-score	.471	.371	.359	.337	.356	.342

Profit

Model	Threshold	"Profit"	Accuracy
Logistic	0.103	484500	0.801
Tree	0.104	490300	0.808
Random Forest	0.109	510600	0.829
Boosted Tree	0.105	508300	0.831
Boosted Tree (Early Stopping)	0.174	494100	0.874
h2o	0.098	503000	0.829



4. Interpreting “Black Box” Learners

Code for Black Box Interpretation Example

"Black Box" Models

Random forests, boosted trees, stacked ensembles are examples of "black box" prediction algorithms

- Produce forecasts, but it's hard to see exactly what goes into prediction rule
  - Many (all?) flexible prediction procedures have this property

One sensible option is just to use such rules to make predictions and not try to (over-) interpret the results

- Good enough for many applications

Common Interpretation Devices

Sometimes one does want to dig a bit deeper into how the prediction rule works

Two simple, commonly used interpretation devices:

- Variable importance measures
  - There are many importance measures – we'll just look at a couple
- Partial dependence plots

Let's look at some examples of these using the random forest model fit in the bank data.

Code for Interpretation in Bank Example

Drop Column Importance

Drop column importance is a simple and robust way to measure feature importance

1. Compute prediction rule using all features and obtain performance measure (typically out-of-sample)
2. For j = 1,…,*p* (where *p* is the total number of features)
   a) Compute prediction rule using all features *excluding* feature j
   b) Obtain performance measure
3. Calculate importance of each feature by comparing performance with feature dropped to performance with all features

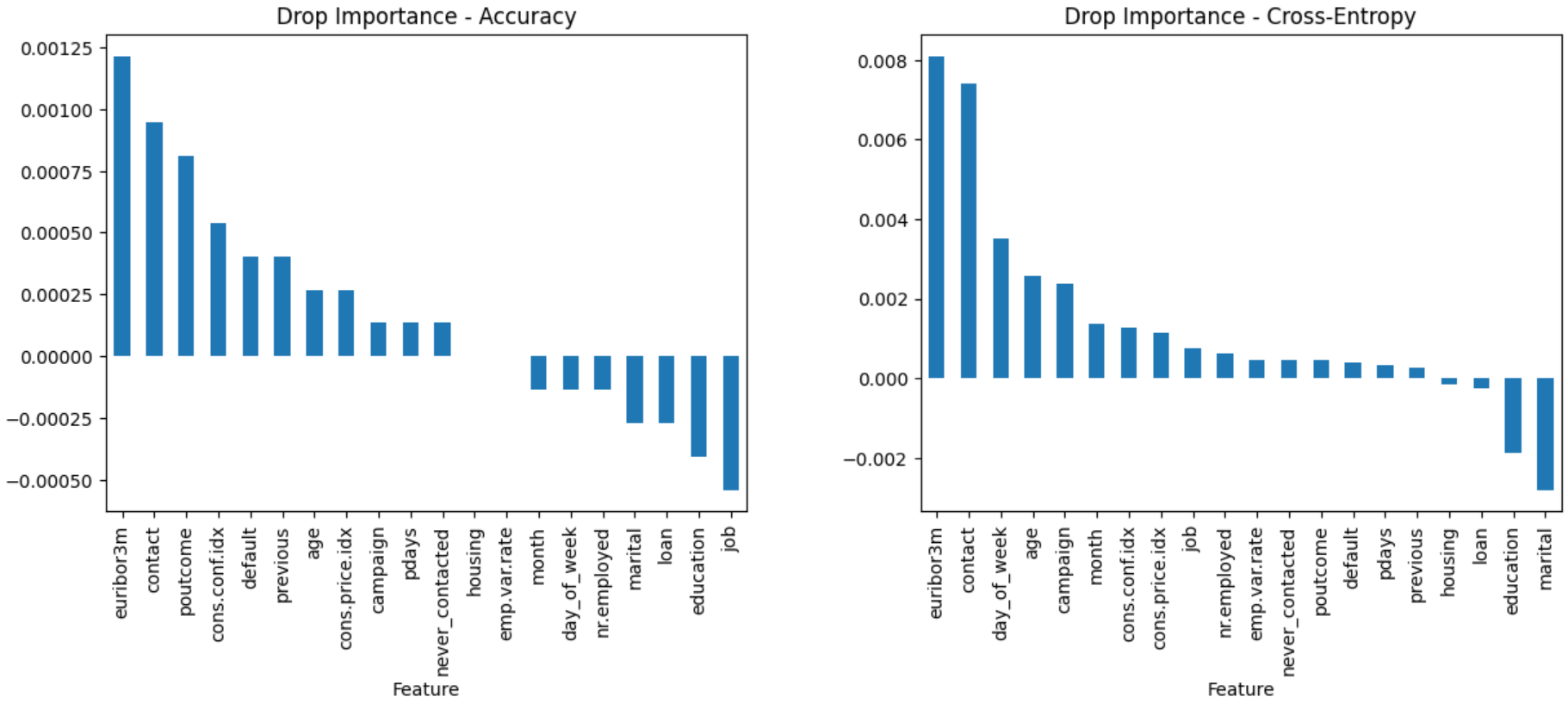
Advantages:

- Robust
- Can be applied to any learner
- Accounts for interactions

Disadvantages:

- Computation
- Highly correlated features

Bank Example: Drop Column Importance



Drop column importance in bank example with importance based on accuracy (left) or cross-entropy (right).

Importance calculated as $\frac{baseline\ performance - performance\ excluding\ variable}{baseline\ performance}$

Permutation Importance

Permutation importance is a simple, relative fast way to measure feature importance

1. Compute prediction rule using all features and obtain performance measure (typically out-of-sample)
2. For j = 1,…,*p* (where *p* is the total number of features)
   a) Randomly permute feature j (leaving all other features unchanged)
   b) Compute prediction rule using data from a)
   c) Obtain performance measure
3. Calculate importance of each feature by comparing performance with randomly permuted features to performance with real features

Can (and usually do) repeat step 2 many times.

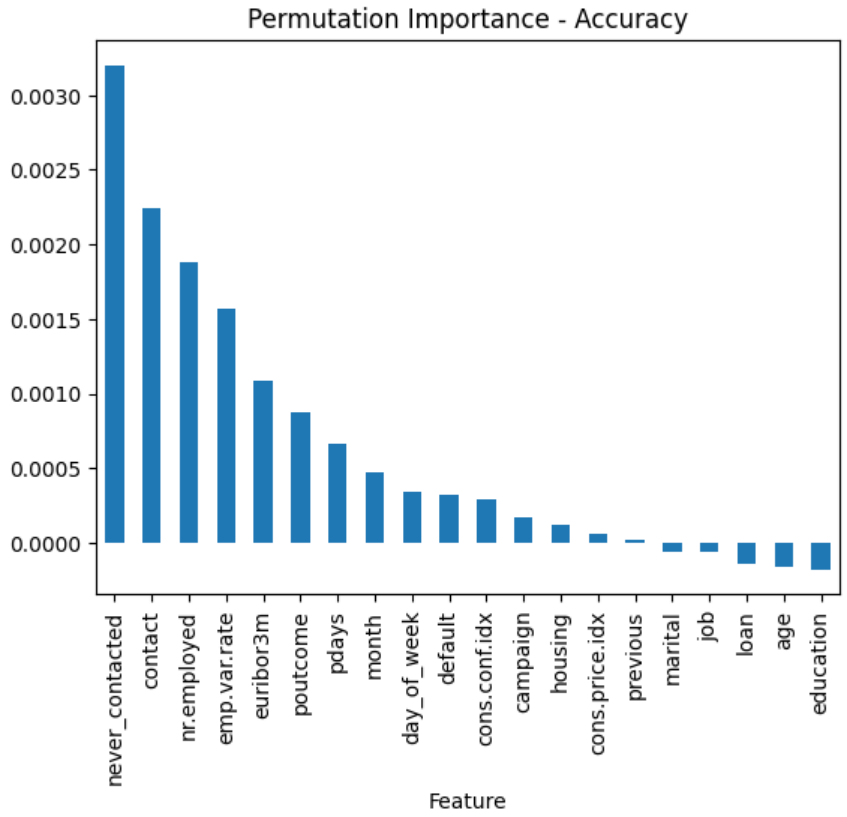
Advantages:

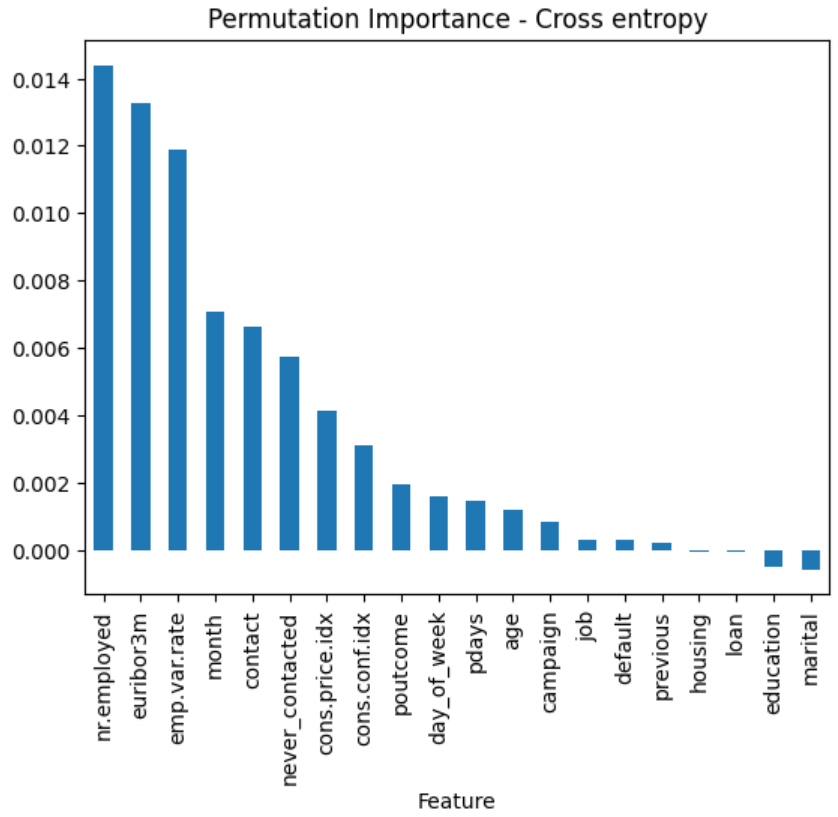
- Relatively fast
- Can be applied to any learner
- Accounts for interactions

Disadvantages:

- Highly correlated features
- Only about the "learned" prediction rule

Bank Example: Permutation Importance





Permutation importance in bank example with importance based on accuracy (left) or cross-entropy (right).

Importance calculated as $baseline\ performance - performance\ permuting\ variable$

Partial Dependence Plots

Any prediction rule is a mapping, $f(\cdot)$, that tells us how to translate a value of our predictor variables, x, to a predicted value for the outcome, $\hat{y}$.

- If the predictor variables were simple (e.g. there was only one predictor), we could understand $f(\cdot)$, no matter how complicated, by just plotting $f(x)$ at many values of x
- Unfortunately, not practical for complex x

Partial dependence plot takes this idea and plots a *summary* of $f(x)$ focusing on one variable at time

- Suppose you are interested in understanding how variable X_j relates to the prediction rule. Let X_{-j} denote all the other variables
- Typical partial dependence plot essentially proceeds as follows:
  1. Choose a set of values for X_j: $(\mathrm{a}_1, \mathrm{a}_2, \ldots, \mathrm{a}_\mathrm{M})$
     - E.g. if X_j denoted age, you might choose values (16,17,18,…,99,100)
  2. For each value $m = 1, \ldots, M$, compute $\bar{f}_j(a_m) = \frac{1}{B}\sum_{i=1}^{B} f(X_j = a_m, X_{-j} = x_{-j,i})$
     - I.e. fix the value of variable X_j, compute the prediction with all other variables set equal to values in a dataset (training, test, or some auxiliary) for all values in that data, take the sample average
  3. Plot the result. I.e. plot $(\bar{f}_j(a_1), \ldots, \bar{f}_j(a_M))$ against $(\mathrm{a}_1, \mathrm{a}_2, \ldots, \mathrm{a}_\mathrm{M})$

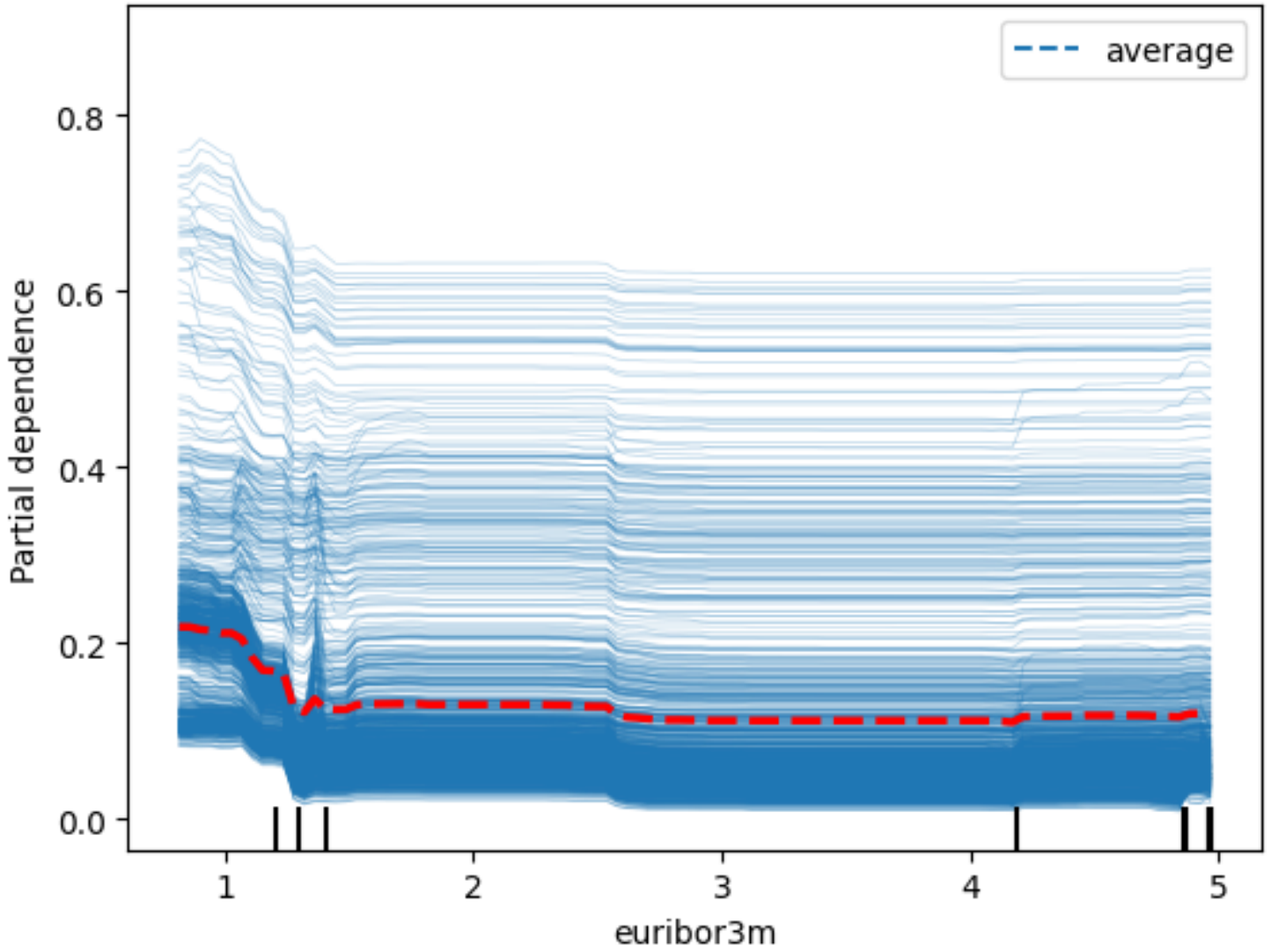
Bank Example: Partial Dependence Plots

Let's see how this works for the variable `euribor3m`

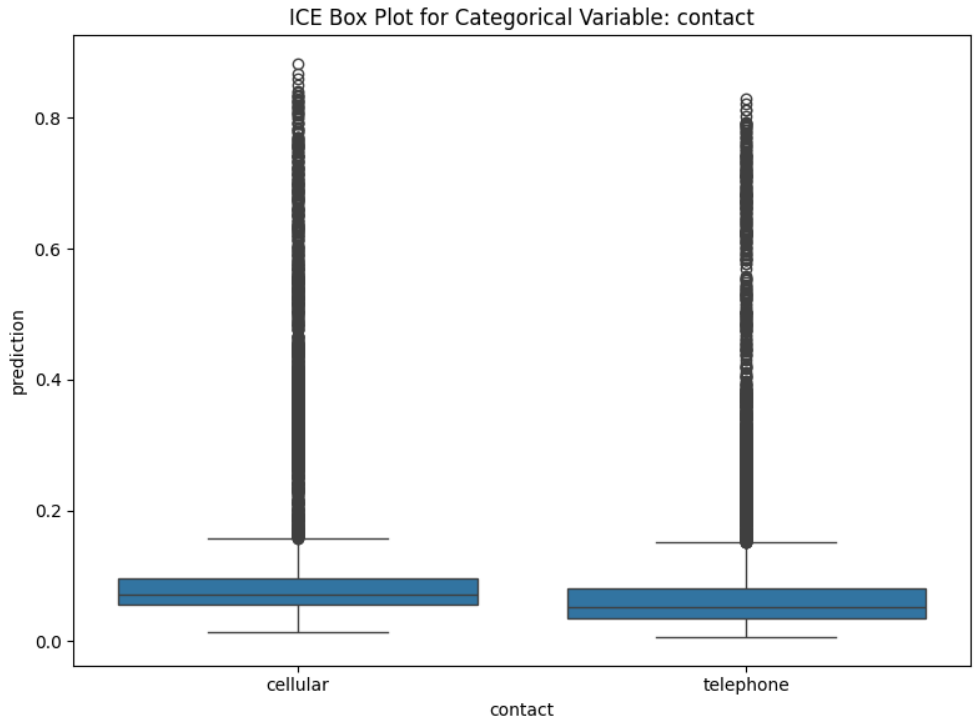
Each line is calculated on the same grid of values for `euribor3m`.

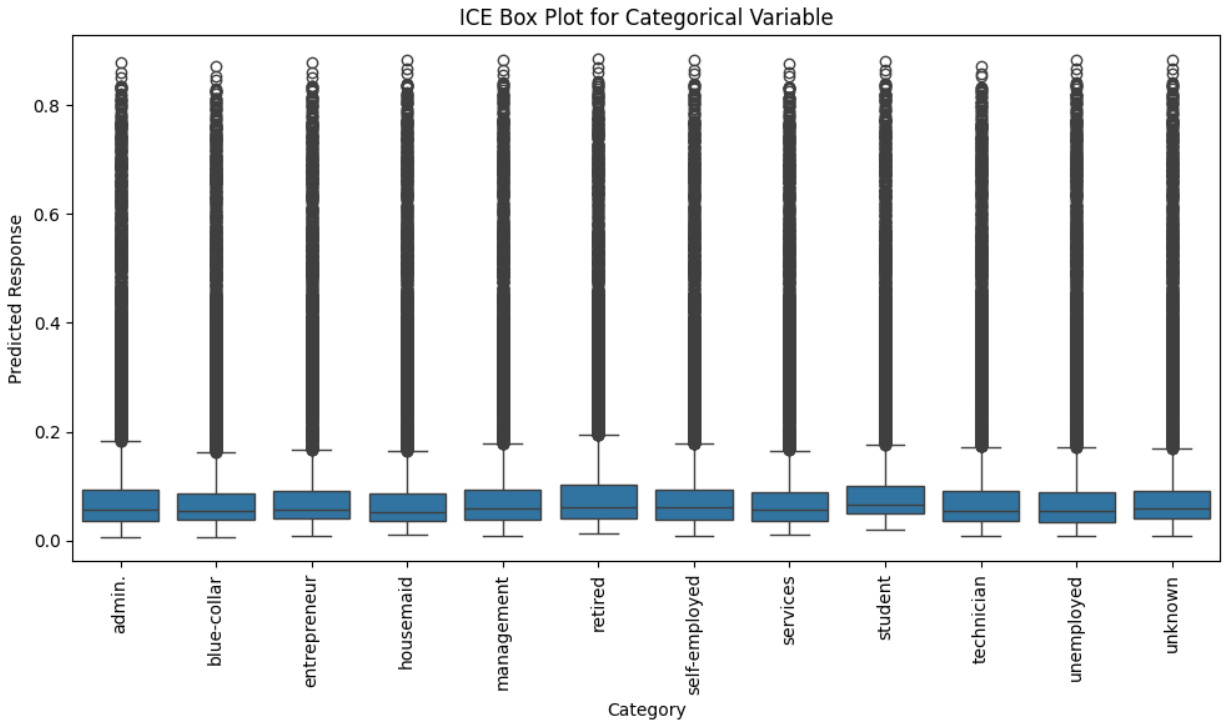
Each blue line corresponds to filling in the values for all X's *except* `euribor3m` with the values from a given observation.

The orange line is then the average of all the blue lines.



Bank Example: Partial Dependence Plots





Things to Remember

Knowing variables that are important in prediction rules tells you very little about mechanisms

- "Correlation does not imply causation"
  - (and lack of correlation does not imply lack of causation)

Think about correlated variables

Think about ultimate goal

Magnitudes of importance measures can be hard to think about

Most measures are variable by variable – can make variables that matter through *interactions* seem less important

Common interpretation devices are high-level summaries – can miss nuance and impact in specific use cases